

Honeypot deployment and containment

Follow-along: play the attacker, then the defender

Overview

Definition: Honeypot

A machine or service deployed to be attacked. It runs no production workload and serves no real users, so it has no legitimate traffic of its own: every connection it receives is either a mistake or an intruder, which is what makes its logs worth reading. A honeypot placed where an attacker will find it first also absorbs the attack that would otherwise have landed on a real host.

A **honeypot** is the one defence in this catalogue that does not try to stop anything. It accepts the attack, keeps a record of it, and gives the attacker somewhere to spend their time that is not your production estate. This lab deploys Cowrie, an SSH honeypot that presents a shell an attacker can type into: it serves a fake filesystem, answers commands, and writes every keystroke it receives to a log, with no real operating system behind any of it.

Deploying the software is the easy half, and this lab spends two of its three parts on the hard half. First, a decoy nobody reaches catches nothing, so you will make hostile traffic arrive at it without moving the address the attacker is aiming for. Then the uncomfortable part: you will have built a machine that an attacker is logged into, sitting on your own network, and the second half of Part 3 is about what that machine is allowed to reach. A honeypot you have not contained is a host you have given away.

Running this lab in the TUI

You drive the lab from the MiniLabs launcher. Clone it if the machine does not have it, and pull if it does, because handouts and lab scripts change together:

```
git clone https://github.com/nzRichie/MiniLab.git ~/MiniLab # first time
cd ~/MiniLab && git pull                                     # already cloned
```

Start the launcher from that directory:

```
cd ~/MiniLab && ./minilabs
```

If it exits with a Docker error, follow the setup in the repository's **README.md** and run it again.

The root menu lists one category per row. Select **Application layer attacks and mitigations**, then **Honeypot deployment and containment**, to reach this lab's actions:

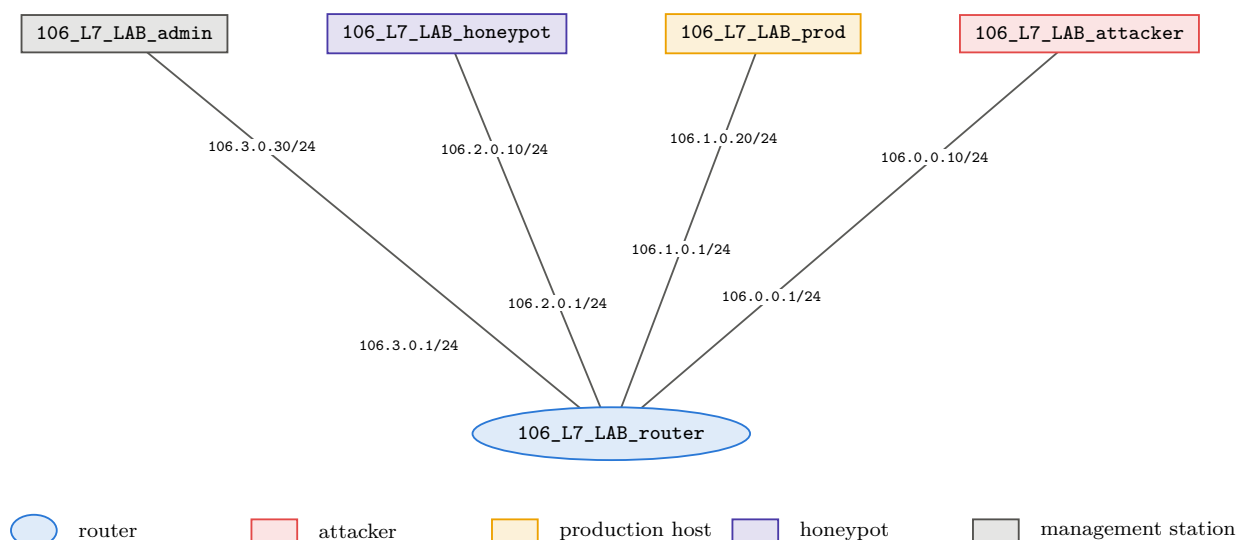
- **Spawn** creates this lab's containers and wiring. Wait for the panel to turn green before you go on. The first spawn of a lab also builds its images; this one installs the honeypot software, so give it a few minutes. Later spawns take seconds.
- **Status** prints the lab's state and its success oracle, so you read the before and after without shelling in. It also says which host a connection from the attacker's network currently reaches, and whether the honeypot can still reach the production host. Status takes over the whole screen: the arrow keys scroll its output, and Enter on **Done** closes it.

- **Shell command for a node** lists every node (attacker, prod, honeypot, admin, router) and the `docker exec -it ...` line that opens a shell on it. Copy the line for the node a step below names into your own terminal. It holds the screen until Enter on **Done**, as Status does.
- **Reset to baseline** removes the decoy you configured and the rules you wrote on the router, and empties the logs this lab counts lines in, returning it to its clean Part 1 state without tearing it down.
- **Teardown** removes the lab.
- **Open handout** opens this document.

Spawn the lab now, then work through Part 1 in your own terminal. **Back** (or Esc) returns to the lab list.

The setup

Four subnets meet at one router. Every segment holds a single host, so there is no switch anywhere in this lab and no shared broadcast domain; the only device that sees traffic between two segments is the router, and it is the only device you will configure.



Each node is labelled with the container name you pass to `docker exec`, and each end of a link with the address on that interface. The same addressing, to copy from:

Role	IP	Subnet	Container
attacker	106.0.0.10	external 106.0.0.0/24	106_L7_LAB_attacker
router	106.0.0.1 (ext) 106.1.0.1 (prod) 106.2.0.1 (honey) 106.3.0.1 (mgmt)	all four	106_L7_LAB_router
production host	106.1.0.20	production 106.1.0.0/24	106_L7_LAB_prod
honeypot	106.2.0.10	honeypot 106.2.0.0/24	106_L7_LAB_honeypot
management	106.3.0.30	management 106.3.0.0/24	106_L7_LAB_admin

The production host runs two services: an `sshd` on port 22, and a web server on port 80 holding an internal document. It also holds a handover note readable by the account you are about

to use. The honeypot has Cowrie installed under `/opt/cowrie` and is not running it. The management station exists so that every defence you write can be checked against a second question: does the administrator still get in?

The router already carries the site's edge filter, written long before anyone thought about decoys. Its policy is that the external network may reach production's SSH port and nothing else. Read it whenever you want to:

```
# 106_L7_LAB_router: the site's existing edge filter
docker exec 106_L7_LAB_router nft list ruleset
```

You attack from `106_L7_LAB_attacker`, holding one credential recovered from an earlier engagement: the account `netadmin` with the password `Rotterdam-4417`. There is no exploit in this lab and no password to guess. Open a shell wherever a step says so with:

```
docker exec -it <container> bash
```

Assessment. This lab is graded on sixteen questions, marked inline below as **Question *N***, each placed at the step whose output it asks about. Answer all sixteen in a separate document as you work through the steps. Every question is answered from what you see on screen at that step.

Part 1: the attack

The attack is two commands long, and that is the point. An attacker holding a working credential for an exposed service does not need anything else, so this part exists to establish what the site looks like before you defend it and to give you the numbers the later parts are measured against.

1. **Log in to the production host and read what is there.** On the attacker:

```
# 106_L7_LAB_attacker: the whole of the attack
docker exec -it 106_L7_LAB_attacker bash
ssh netadmin@106.1.0.20                # password: Rotterdam-4417
hostname
id
uname -a
cat /srv/internal/handover.txt
```

Write down what `id` and `uname -a` report. You will run the same three commands again in Part 2 against something that is not this machine, and the differences between the two sets of output are what a careful attacker would use to tell them apart.

2. **Try the other service.** Still on the attacker, and still in that SSH session's host's own network, ask production's web server for the same internal document:

```
# 106_L7_LAB_attacker: run this in a second terminal, outside the SSH session
docker exec 106_L7_LAB_attacker curl -m 10 http://106.1.0.20/internal.html
```

Question 1. Your SSH session connected, and this fetch produced no output and no error

message until `curl` gave up. Explain, from the router's edge filter, why the fetch behaves this way rather than returning `Connection refused`, and state what a `Connection refused` would have told you about the production host that the timeout does not.

Question 2. On the production host, `/var/log/prod-sshd.log` records every authentication the real `sshd` handled. Count the lines in it that record a successful password login, and state which of them are yours. Keep this number: Part 2 is measured against it.

```
# 106_L7_LAB_prod: the real host's authentication record
docker exec 106_L7_LAB_prod grep 'Accepted password' /var/log/prod-sshd.log
```

Part 2: put a decoy in the way

The defence has two halves and neither works alone. First you configure and start the honeypot, which gives you a service that records what is done to it. Then you make the attacker's traffic arrive at it, without changing the address the attacker is aiming for and without cutting off the administrators who need the real host.

Nothing here removes the attacker's credential or patches anything. The production host keeps running exactly the service it was running in Part 1, and the account keeps working. What changes is which machine answers.

Configure the decoy

Definition: Emulated shell

A program that presents a command prompt, accepts commands, and prints plausible replies without running any of them on a real operating system. Cowrie's is backed by a filesystem image built from a real Linux install, so directory listings and file contents look ordinary, but nothing an attacker types is ever executed by a kernel. The gap between the two is what limits how long the illusion survives an attacker who probes it.

Cowrie is installed at `/opt/cowrie` and runs as an unprivileged account, also called `cowrie`. It reads its defaults from the `cowrie.cfg.dist` bundled inside the installed package and reads `etc/cowrie.cfg` on top of them, so the file you write only has to carry what differs. Open a shell as the account that runs it:

```
# 106_L7_LAB_honeypot: work as the account Cowrie runs under, in its state
  directory
docker exec -it -u cowrie -w /opt/cowrie 106_L7_LAB_honeypot bash
export PATH=/opt/cowrie-venv/bin:$PATH
```

That shipped file lists every option Cowrie has, each with its default and a comment saying what it changes. It is the first place to look when you want the decoy to behave differently, and a question later in this part is easier to answer if you have read it.

```
# 106_L7_LAB_honeypot: the defaults, with every option documented
less /opt/cowrie-venv/lib/python3.11/site-packages/cowrie/data/etc/cowrie.cfg.dist
```

Write the operator config. `hostname` is what the **emulated shell** calls itself at the prompt and in `uname`; setting it to the name of the host the decoy is standing in for is the difference between

a decoy and a sign saying "decoy".

```
# 106_L7_LAB_honeypot: in the shell you already have open
cat > etc/cowrie.cfg <<'EOF'
[honeypot]
hostname = prod
listen_endpoints = tcp:2222:interface=0.0.0.0

[ssh]
listen_endpoints = tcp:2222:interface=0.0.0.0

[telnet]
enabled = false

[output_jsonlog]
enabled = true
logfile = ${honeypot:log_path}/cowrie.json
EOF
```

Then decide which credentials the decoy accepts. `etc/userdb.txt` holds one rule per line as `username:unused:password`, and the first match wins. A `*` in either field matches anything.

```
# 106_L7_LAB_honeypot: accept exactly the credential the attacker already holds
cat > etc/userdb.txt <<'EOF'
netadmin:x:Rotterdam-4417
EOF
```

Question 3. Accepting exactly one credential is one choice; a single line reading `*:x:*`, which accepts any username with any password, is the other common one. Take an attacker who tries a password they already know is wrong before trusting the host. State what each of the two configurations tells that attacker, and what each one costs you in sessions you never get to record.

Start it, and confirm it is listening:

```
# 106_L7_LAB_honeypot: in the shell you already have open
cowrie start
netstat -tln | grep 2222
```

Question 4. Cowrie refuses to run as `root`, and the port it listens on is 2222 rather than 22. Explain the connection between those two facts. Then state what it forces the rule you are about to write on the router to do that it would not have had to do if the decoy were listening on 22.

Send hostile traffic to it

Definition: Destination NAT

Rewriting the destination address, and optionally the destination port, of a packet as it passes through a router, so it is delivered to a different host than the sender addressed. The sender is told nothing: it sees a connection to the address it dialled, answered normally. In nftables this is the `dnat` statement, and it is only available in a chain of type `nat`.

Definition: The prerouting hook

The point in the kernel's packet path where a packet is seen after it arrives on an interface and before the router decides where to send it. A **destination NAT** rule has to sit here, because rewriting the destination after the routing decision would send the packet to the host the old address selected. A rule at **prerouting** changes the address first, and the routing decision is then made on the new one.

Write the redirect on the router. Two rules, and their order is the policy:

```
# 106_L7_LAB_router: redirect hostile SSH, exempt the management network
docker exec -it 106_L7_LAB_router bash
nft add table ip deception
nft 'add chain ip deception prerouting { type nat hook prerouting priority dstnat
; }'
nft add rule ip deception prerouting ip saddr 106.3.0.0/24 accept
nft add rule ip deception prerouting ip daddr 106.1.0.20 tcp dport 22 counter dnat
to 106.2.0.10:2222
```

In a **nat** chain, **accept** means "apply no translation to this packet", not "let this packet through a firewall". The first rule therefore sends the management network to the real host, and it only does so because it sits above the rule that would otherwise have rewritten it.

Question 5. Predict what happens to the administrator's next SSH session if those two rules are written in the other order, with the redirect first. Then say what the administrator would see on screen, and why nothing in the router's own output would look like an error.

The redirect changes a packet's destination before the edge filter's forward rules examine it, so the traffic now arriving is external-to-honeypot and no longer matches the rule that admitted external-to-production SSH. Add the permit it needs:

```
# 106_L7_LAB_router: in the shell you already have open
nft add rule inet edge forward ip saddr 106.0.0.0/24 ip daddr 106.2.0.10 tcp dport
2222 counter accept
```

Run the attack again

Repeat Part 1 exactly: same address, same credential, same commands.

```
# 106_L7_LAB_attacker: the identical session from Part 1
docker exec -it 106_L7_LAB_attacker bash
ssh netadmin@106.1.0.20 # password: Rotterdam-4417
hostname
id
uname -a
cat /srv/internal/handover.txt
exit
```

Question 6. The credential worked and the prompt looks the same, but you are not on the production host. Compare this session's output against what you wrote down in Part 1 and name three things that differ. For each one, say whether an attacker who was not looking for a honeypot would have noticed it.

Now read the two logs. On the production host, the authentication record you counted in Question 2; on the honeypot, Cowrie's JSON log, one event per line:

```
# 106_L7_LAB_prod: has the real host seen anything?
docker exec 106_L7_LAB_prod grep -c 'Accepted password' /var/log/prod-sshd.log
```

```
# 106_L7_LAB_honeypot: what the decoy recorded
docker exec 106_L7_LAB_honeypot grep -o '"eventid": "[^"]*"'
/opt/cowrie/var/log/cowrie/cowrie.json | sort | uniq -c
docker exec 106_L7_LAB_honeypot grep -o '"input": "[^"]*"'
/opt/cowrie/var/log/cowrie/cowrie.json
```

Question 7. State the count of successful password logins on the real production host now, next to the number you recorded in Question 2. Then read the packet counter on your `dnat` rule (`nft list table ip deception`) and give its value. Explain what those three numbers together establish that either one alone would not.

Question 8. From the event summary, name the event Cowrie writes when an attacker types a command, and give the number of them your log holds next to the number of command lines you sent. Then name the event that records the credential itself, and say why a defender would rather have that line than a firewall log showing a connection to port 22.

Check the other half, the one that stops you passing this lab by breaking it:

```
# 106_L7_LAB_admin: the administrator's access must survive the defence
docker exec -it 106_L7_LAB_admin bash
ssh netadmin@106.1.0.20          # password: Rotterdam-4417
hostname
cat /srv/internal/handover.txt
exit
```

RFC 4253 section 4.2 requires both peers on a new SSH connection to send an identification string, the `SSH-2.0-...` line naming the software. It does not say which peer sends first, and both servers in this lab send theirs as soon as the connection opens, so a probe that connects and reads one line gets an answer from each.

```
# 106_L7_LAB_admin, then 106_L7_LAB_attacker: same probe, same address, two answers
# 'sleep 1 |' holds the connection open; nc hangs up as soon as its own input ends
docker exec 106_L7_LAB_admin sh -c 'sleep 1 | nc -w 8 106.1.0.20 22'
docker exec 106_L7_LAB_attacker sh -c 'sleep 1 | nc -w 8 106.1.0.20 22'
```

Question 9. Record the identification string each probe returned. Name the part of the two strings that differs, and, given that both servers are behaving as RFC 4253 requires, say what an attacker who already knows what this site's production `sshd` answers would conclude from the pair. Then state what you would change in the decoy's own configuration to close that gap.

Part 3: the decoy is now your problem

You have a machine on your network that an attacker is logged into, and you put it there. The question this part asks is what that machine is allowed to reach.

Cowrie's shell is emulated, so most of what an attacker types is answered by a Python function and touches nothing. Its file-transfer commands are the exception, and deliberately so: when an attacker runs `wget`, Cowrie performs the download for real and keeps the file, because capturing the malware an intruder brings with them is one of the reasons to run a honeypot at all. The request goes out over the network, from the honeypot's own address.

Reach production from inside the decoy

Question 10. Before you run it, predict whether `wget http://106.1.0.20/internal.html` typed at the decoy's prompt can return the document, given that the edge filter drops HTTP from the external network to production. Name the machine that would make the request if it succeeded, and the source address production would see.

```
# 106_L7_LAB_attacker: from inside the decoy's shell
docker exec -it 106_L7_LAB_attacker bash
ssh netadmin@106.1.0.20                # password: Rotterdam-4417
wget http://106.1.0.20/internal.html
cat internal.html
exit
```

```
# 106_L7_LAB_prod: the web server's record of who asked
docker exec 106_L7_LAB_prod cat /var/log/lighttpd/access.log
```

Question 11. Give the source address and the `User-Agent` string on that request. Explain what an incident responder learns from those two fields together that the attacker's own address would not have told them, and name one thing about the attacker that this log entry does not contain.

Question 12. The edge filter drops HTTP from 106.0.0.0/24 to 106.1.0.0/24, and the fetch you just ran was HTTP to 106.1.0.20. Explain why that rule did not stop it. Your answer should name the field the rule matches on and what its value was for this packet.

Contain it

Definition: Connection tracking

The kernel's record of the connections passing through a router: for each one it remembers the addresses, ports, and direction of the packet that opened it. A rule can then match on a packet's relation to that record rather than on its contents. `ct state new` is a packet that starts a connection the router has not seen before; `ct state established` is a packet belonging to one it has already seen a reply on, in either direction.

A decoy answers connections; it has no reason of its own to start any, so every connection it opens is one an attacker caused. That is the whole policy, and **connection tracking** is what lets you write it in one rule without also breaking the attacker's session, whose return packets are sourced from the honeypot too.

The edge filter's chain already accepts established and related traffic as its first rule. Read it before you add anything, and note where your rule will land:


```
# 106_L7_LAB_router: the chain you are about to append to
docker exec 106_L7_LAB_router nft -a list chain inet edge forward
```

```
# 106_L7_LAB_router: the decoy answers sessions, and starts none
docker exec 106_L7_LAB_router nft add rule inet edge forward ip saddr 106.2.0.0/24
ct state new counter drop
```

Question 13. The chain's first rule accepts `ct state established,related`. Suppose that rule were deleted while yours stayed. Work through the attacker's SSH session into the decoy one direction at a time: for the packets travelling each way, say which rule in the chain they match. Then state what decides whether the return packets pass once you reach the end of the list.

Run the pivot again. Give it longer than feels necessary; a dropped packet produces no answer, so the fetch has to wait for its own timeout.

```
# 106_L7_LAB_attacker: the identical pivot, now contained
docker exec -it 106_L7_LAB_attacker bash
ssh netadmin@106.1.0.20                # password: Rotterdam-4417
wget http://106.1.0.20/internal.html
cat internal.html
exit
```

```
# 106_L7_LAB_prod: did the web server serve it a second time?
docker exec 106_L7_LAB_prod cat /var/log/lighttpd/access.log
```

Question 14. You scoped the rule to the whole honeypot subnet rather than to production alone. Using an address from this lab, name one thing the narrower rule `ip saddr 106.2.0.0/24 ip daddr 106.1.0.0/24 ct state new drop` would still have let the decoy do, and say why that matters for a host an attacker is logged into.

Question 15. Confirm the decoy is still doing its job by reading its command log again, then confirm the administrator still reaches the real host. State the evidence for each. A defence that stops the pivot by stopping the honeypot has not passed this lab.

Question 16. This deployment ends with the attacker's commands recorded and the production host untouched. State one capability the attacker still has here that they would not have had if the site had run no honeypot at all, and name the rule in your configuration that bounds it.

Checks: what “done” means

- **Attack works:** with no decoy and no redirect, the attacker's SSH session reaches 106.1.0.20 itself, reads `/srv/internal/handover.txt`, and adds a line to `/var/log/prod-sshd.log`.
- **Redirect holds:** the same session lands on the honeypot, the handover note is not there, `/var/log/prod-sshd.log` gains nothing, Cowrie's JSON log holds one `cowrie.login.success` and one `cowrie.command.input` per command line you sent, and the management station still reaches the real host.

- **Containment holds:** the same `wget` from inside the decoy times out, production's access log gains no second entry from 106.2.0.10, Cowrie still records the attempt, and the management station still reaches the real host.

`scripts/status.sh` reports which host a session from the attacker's network reaches, whether the honeypot can still open a connection to production, and whether the management station still gets through, so you can read before and after without shelling in. To undo your work, `scripts/reset.sh` removes the decoy's configuration and both router rules and empties the two logs, back to the Part-1 baseline. `scripts/teardown.sh` removes the lab.

Safety. Everything here lives on one isolated router and four point-to-point links with no route off the host. The addresses are real public address space, chosen so the honeypot's own outbound checks treat them as ordinary internet destinations, and no container in this lab can route anywhere but the lab's own hosts. The credential, the handover note and the internal document exist only inside these containers.

When you are done, tear the lab down. Choose **Teardown** in the TUI, or run `scripts/teardown.sh` directly, to remove the containers and their wiring so nothing is left running on your host. **Reset to baseline** only rewinds the decoy and your router rules; only **Teardown** deletes the lab.